

PROJECT WORK – III
SYNOPSIS REPORT

**ARBITER: An Adaptive Multi-Agent System for Iterative Task Generation,
Validation and Refinement**

by

Gautam Lasgotra
Entry No.: 23BCS032

Chirag Attri
Entry No.: 23BCS024

Aniket Kundal
Entry No.: 23BCS015

Under the Guidance
Of
Dr. Sonika Gupta

Submitted in partial fulfilment of the requirements for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE & ENGINEERING



SHRI MATA VAISHNO DEVI UNIVERSITY, KATRA

(School of Computer Science & Engineering)

JAMMU & KASHMIR – 182 320

Session 2026-27 (Odd)

1. Title of the Project

ARBITER: An Adaptive Multi-Agent System for Iterative Task Generation, Validation and Refinement — with a Budget-Matched Evaluation of When Multi-Agent Refinement Helps.

2. Name(s) of the Student(s) with Entry Number(s)

Gautam Lasgotra (23BCS032) · Chirag Attri (23BCS024) · Aniket Kundal (23BCS015)

3. Name of the Guide (with Designation & Department)

Dr. Sonika Gupta, Assistant Professor, School of Computer Science & Engineering, SMVDU Katra.

4. Introduction and Background of the Study

Large Language Models produce plausible output but cannot reliably verify their own work. A widely studied response is *multi-agent refinement*: one agent produces a solution, another inspects or tests it, the feedback is returned, and the solution is revised. Systems such as Self-Refine, Reflexion and AgentCoder report improvements from this pattern, and frameworks such as ChatDev and MetaGPT extend it to role-specialised teams.

Two recent findings complicate that picture. First, Huang et al. (ICLR 2024) showed that without *external* feedback, self-correction does not improve results and frequently degrades them — implying that the verifier, not the critic, is the active ingredient. Second, budget-matched studies (Tran & Kiela, 2025) found that once the number of reasoning tokens is held equal, a single agent matches or exceeds multi-agent systems on reasoning tasks, concluding that reported multi-agent advantages are better explained by unaccounted computation than by architecture.

Meanwhile, a newer line of work proposes *adaptive* orchestration — selecting agents and workflow depth per task so that effort is spent only where it is needed. This project builds such a system and then applies to it the one test its own justification demands.

5. Objectives of the Project

1. Design and implement an adaptive multi-agent system that accepts an unseen natural-language task at runtime, selects appropriate agent roles, generates a solution, validates it using executable tools, returns feedback, and refines iteratively under explicit budget limits.
2. Implement independent validation using deterministic tools (program execution, query execution, answer matching) rather than relying on an LLM to judge its own output.
3. Implement baseline conditions — including a **budget-matched** single agent — so that any observed benefit can be attributed to architecture rather than to extra computation.
4. Construct a benchmark spanning three task families that differ in the strength of the available automatic verifier, and measure correctness, iterations, agents used, token usage, latency and cost.
5. Provide a web interface that exposes the live execution trace, so that agent decisions, validation results and resource consumption are observable rather than hidden.

6. Problem Statement / Research Question

Adaptive multi-agent orchestration is justified on the grounds of computational efficiency — spending agents only when a task needs them. That is a claim about cost. Yet existing adaptive systems are evaluated against the strongest available baseline rather than against a single agent given an *equal token budget*, which is precisely the comparison that deflated earlier multi-agent claims.

Primary research question. Under a matched token budget, does adaptive multi-agent refinement improve task correctness over (a) a budget-matched single agent and (b) fixed multi-agent pipelines?

Secondary questions. (i) Does any observed benefit depend on the strength of the automatic verifier available for the task? (ii) Does combining tool-based validation with an LLM critic reduce undetected errors compared with LLM-only review? (iii) How many refinement iterations remain useful before further iterations stop paying for themselves?

The contribution claimed is a **measurement and an evaluation methodology**, not a novel architecture. Multi-agent collaboration and adaptive orchestration are both established prior art and are treated as such throughout.

7. Methodology / Approach

System. A task analyser classifies the incoming task; a planner emits a workflow specification selecting roles from a fixed library (generator, test designer, critic, repair); the orchestrator runs a bounded generate → validate → decide loop. Validation is performed first by deterministic tools and only then by an LLM critic, whose verdict is recorded but never treated as authoritative. For tasks supplied at runtime, a test-designer agent first derives executable tests from the task description.

Termination. The loop stops on acceptance, or on any of: maximum iterations, LLM calls, token budget, wall-clock limit, two iterations without score improvement, or a repeated identical output. Every termination reason is logged; unbounded loops are structurally impossible.

Evaluation. Five conditions are compared on a frozen benchmark:

Condition	Description
A	Single agent, single attempt
A+	Single agent given the same token budget as condition D, spent on repeated sampling with majority voting
B	Generator ↔ validator refinement loop
C	Fixed pipeline: generator → critic → repair
D	ARBITER — planner selects roles and iteration limit per task

All conditions share one engine and differ only by configuration, so differences cannot arise from implementation divergence. Every metric is computed from logged execution traces.

8. Tools, Technologies, or Datasets to be Used

Component	Choice
Language / backend	Python 3.10+, FastAPI, Pydantic v2
Orchestration	Custom state machine (no heavyweight agent framework, so the logic under study remains inspectable)
Language model	Google Gemini (Flash-Lite tier) behind a provider abstraction; secondary provider for cross-model validation experiments
Execution sandbox	Isolated subprocess with timeout, restricted environment and resource limits
Storage / logging	JSONL execution traces, SQLite, on-disk response cache for reproducibility
Interface	Server-Sent Events streaming to a browser client
Datasets	Code: HumanEval+/MBPP+ (EvalPlus) · SQL: Spider · Mathematical reasoning: GSM8K

9. Expected Outcomes / Deliverables

- A working adaptive multi-agent system that accepts unseen runtime tasks, performs real model calls, validates with executable tools, refines iteratively, and terminates under an explicit budget.
- A web interface showing the live execution timeline, agent roles, validation results and resource consumption, with a replay mode for stored runs.
- A frozen benchmark across three task families with differing verifier strength.
- A comparative results table across the five conditions, reporting correctness alongside tokens consumed per task solved.

- An analysis of failure modes classified using an established multi-agent failure taxonomy.
- Reproducible experiment logs, a documented architecture, and a project report.

Expected findings are stated as hypotheses only and will be reported from measurement, whatever the outcome. We hypothesise that benefit tracks verifier strength — largest where executable tests exist and smallest where only a final answer can be checked — and that most improvement occurs within the first two refinement iterations. A result showing that adaptive orchestration does *not* outperform a budget-matched single agent would be a valid and reportable finding.

10. Timeline of Work

Phase	Dates	Work
Phase 0 — Literature and design	22 – 28 Sep 2026	Literature survey, prior-art matrix, research-gap analysis, architecture and evaluation design, technology selection
Phase 1 — Core system	29 Sep – 10 Oct 2026	Orchestration loop, budget governor, provider abstraction with caching, sandboxed execution, validators, test-designer agent, baselines A/A+/B/C, web interface
Mid-semester evaluation	19 – 21 Oct 2026	Demonstration of the refinement loop and initial comparative results
Phase 2 — Research depth	11 – 31 Oct 2026	Adaptive planner (condition D), SQL and mathematical-reasoning families, cross-model validation, budget-matched protocol
Phase 3 — Experiments	1 – 10 Nov 2026	Frozen benchmark sweep, metrics, ablation on iteration count and validator type, failure analysis, deployment, report
Final evaluation	23 – 27 Nov 2026	Report, demonstration and viva

11. References

1. Madaan, A. et al. (2023). *Self-Refine: Iterative Refinement with Self-Feedback*. NeurIPS 2023.
2. Shinn, N. et al. (2023). *Reflexion: Language Agents with Verbal Reinforcement Learning*. NeurIPS 2023.
3. Huang, D. et al. (2023). *AgentCoder: Multi-Agent-based Code Generation with Iterative Testing and Optimisation*. arXiv:2312.13010.
4. Huang, J. et al. (2024). *Large Language Models Cannot Self-Correct Reasoning Yet*. ICLR 2024. arXiv:2310.01798.
5. Kamoi, R. et al. (2024). *When Can LLMs Actually Correct Their Own Mistakes? A Critical Survey of Self-Correction of LLMs*. TACL. arXiv:2406.01297.
6. Cemri, M. et al. (2025). *Why Do Multi-Agent LLM Systems Fail?* NeurIPS 2025. arXiv:2503.13657.
7. Tran, K. and Kiela, D. (2025). *Compute-matched comparison of single-agent and multi-agent LLM systems*.
8. Wu, Q. et al. (2023). *AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation*. arXiv:2308.08155.
9. Hong, S. et al. (2023). *MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework*. arXiv:2308.00352.
10. Qian, C. et al. (2023). *ChatDev: Communicative Agents for Software Development*. arXiv:2307.07924.

11. *From Static Templates to Dynamic Runtime Graphs: A Survey of Workflow Optimization for LLM Agents.* arXiv:2603.22386.
12. Liu, J. et al. (2023). *Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of LLM Code Generation (EvalPlus).* NeurIPS 2023.
13. Yu, T. et al. (2018). *Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL.* EMNLP 2018.
14. Cobbe, K. et al. (2021). *Training Verifiers to Solve Math Word Problems (GSM8K).* arXiv:2110.14168.

Signatures of the Students

Gautam Lasgotra

Entry No.: 23BCS032

Chirag Attri

Entry No.: 23BCS024

Aniket Kundal

Entry No.: 23BCS015

Signature of the Guide

Dr. Sonika Gupta

Assistant Professor

School of Computer Science & Engineering

SMVDU, Katra